

CSE 312 · OPERATING SYSTEMS



GÜVENLİK

Security · Detaylı Konu Anlatım Rehberi

Ders: İşletim Sistemleri (Operating Systems)

Kaynak: Tanenbaum & Bos, *Modern Operating Systems*, 5. Baskı

Dönem: Spring 2026 · **Öğretim Üyesi:** Prof. Dr. Yakup Genç

Gebze Teknik Üniversitesi · Bilgisayar Mühendisliği

Ders 9 — Güvenlik · Sınava Hazırlık Notu

İçindekiler

- 1 Güvenliğe Giriş ve Temel Kavramlar** CIA, tehditler, saldırganlar, OS'in rolü, authentication
- 2 Kriptografinin Temelleri** Secret-key, public-key, digital signature, hashed passwords + salt
- 3 Koruma Mekanizmaları** Protection domains, protection matrix, ACL, capabilities
- 4 Güvenilir Sistemler ve Formal Modeller** TCB, reference monitor, Bell-La Padula, Biba
- 5 Covert Channels (Gizli Kanallar)** Confinement problem, steganography
- 6 Kimlik Doğrulama (Authentication)** Şifreler, challenge/response, token, smart card, biometrics
- 7 Saldırı Teknikleri** Trojan, trap door, login spoofing, buffer overflow, code injection, Heartbleed
- 8 Kötücül Yazılımlar (Malware)** Virüsler, solucanlar, spyware, rootkit türleri
- 9 Savunma Mekanizmaları** Firewall, virus scanner, code signing, jailing, sandboxing, Java security
- 10 Sertifikalı Sistemler ve Askeri Sınıflandırma** Orange Book, Common Criteria, multi-level security
- 11 Loglama (Logging) ve Sonuç** Ne loglanmalı, Solaris BSM, gerçek hedef: uygulamalar

Bu rehber, hocanın 102 slaytlık sunumundaki her konuyu slayt sırasına sadık kalarak, derinlemesine Türkçe anlatımla işler. Teknik terimler İngilizce bırakılmıştır.

1

Güvenliğe Giriş ve Temel Kavramlar

Security & Operating Systems · CIA · Threats · Authentication

İşletim sistemi güvenliği (**OS security**), bir bilgisayar sistemini istenmeyen erişimlere ve eylemlere karşı koruma disiplini. Bu bölümde "güvenlik" kavramını tanımlıyor, işletim sisteminin sistem güvenliğine nasıl katkı sağladığını ve güvenli bir sistem tasarlarlarken OS'ten neler beklediğimizi inceliyoruz.

1.1 İşletim Sistemi Güvenliği Nedir?

Bir işletim sistemi güvenliği tartışmasının çıkış noktası üç sorudur: (1) *OS security tam olarak nedir?* (2) *İşletim sistemleri sistem güvenliğine nasıl katkıda bulunur?* (3) *Güvenli bir sistem geliştirmeye çalışıyorsak, işletim sisteminden ne talep etmeliyiz?* Bu dersin odağı, OS'in güvenlik için **ne yapabileceği**, **ne yapması gerektiği** ve **ne yaptığı** üzerinedir.

TANIM — GÜVENLİK (SECURITY)

Gayriresmî: Güvenlik, yetkisiz varlıkların (**unauthorized entities**) sizin istemediğiniz şeyleri yapmasını engellemektir.

Resmî (CIA üçlüsü): Güvenlik üç temel hedefle tanımlanır — **Confidentiality** (gizlilik), **Integrity** (bütünlük) ve **Availability** (erişilebilirlik).

Bu üç hedef literatürde **CIA üçlüsü** olarak bilinir ve neredeyse tüm güvenlik tartışmalarının çatısını oluşturur. **Confidentiality**, verinin yalnızca yetkili kişilerce görülmesini; **Integrity**, verinin izinsiz değiştirilememesini; **Availability** ise sistemin ve verinin gerektiğinde erişilebilir olmasını ifade eder. İşletim sisteminin bu hedeflere katkısı, dersin geri kalanının ana konusudur.

Güvenliğin Üç Temel Hedefi — CIA Üçlüsü

Confidentiality

Gizlilik

Veriye yalnızca yetkili kişiler erişebilir



Integrity

Bütünlük

Veri izinsiz değiştirilemez / bozulamaz



Availability

Erişilebilirlik

Sistem ve veri gerektiğinde hazır olmalı



Şekil 1.1 — Confidentiality, Integrity, Availability: güvenliğin üç sütunu.

1.2 Tehditler ve Hedefler (Security Goals and Threats)

Her güvenlik hedefinin karşısında onu ihlal etmeye yönelik bir **tehdit** (*threat*) bulunur. Confidentiality'nin karşısında verinin ifşası (*exposure / data theft*), Integrity'nin karşısında kurcalama (*tampering*) ve Availability'nin karşısında hizmet kesintisi (*denial of service*) yer alır. Güvenlik tasarımı, bu hedef-tehdit eşleşmelerini sistematik biçimde ele almakla başlar.

Güvenlik Hedefi (Goal)	İlgili Tehdit (Threat)	Örnek
Confidentiality (Gizlilik)	Data exposure / disclosure	Bir saldırganın gizli dosyaları okuması
Integrity (Bütünlük)	Tampering / alteration	Banka bakiyesinin izinsiz değiştirilmesi
Availability (Erişilebilirlik)	Denial of Service (DoS)	Sunucunun isteklere yanıt veremez hâle gelmesi

1.3 Saldırganlar (Intruders)

Güvenlik tehditlerinin önemli bir kısmı insan kaynaklıdır. Tanenbaum, saldırganları (*intruders*) motivasyon ve yetkinliklerine göre dört ortak kategoride toplar:

- **Teknik olmayan kullanıcıların gelişigüzel meraklı kurcalamaları** (*casual prying by nontechnical users*): Örneğin ortak bir bilgisayarda başkasının e-postasını okumak.
- **İçeridekilerin gözetlemesi** (*snooping by insiders*): Sisteme zaten erişimi olan çalışanların yetkilerini kötüye kullanması.
- **Para kazanmaya yönelik kararlı girişimler** (*determined attempts to make money*): Profesyonel siber suçlular.
- **Ticari veya askeri casusluk** (*commercial or military espionage*): En yetkin ve kaynaklı saldırgan sınıfı.

SINAV İPUCU

Prof. Genç'in tarzında "saldırgan kategorilerini sıralayıp her birine örnek veriniz" türü bir **kategorizasyon** sorusu gelebilir. Dört kategoriyi *motivasyon* (merak → para → casusluk) ve *yetkinlik* ekseninde sıralı ezberleyin.

1.4 Kazara Veri Kaybı (Accidental Data Loss)

Tüm tehditler kötü niyetli değildir. Verinin önemli bir kısmı **kaza sonucu** kaybolur ve iyi bir güvenlik tasarımı bunu da hesaba katmalıdır. Kazara veri kaybının üç ortak nedeni şunlardır:

- **Doğal afetler / "Acts of God"**: Yangın, sel, deprem, savaş, ayaklanma; hatta yedekleme bantlarını kemiren fareler gibi beklenmedik olaylar.
- **Donanım veya yazılım hataları**: CPU arızaları, okunamayan disk/bantlar, telekomünikasyon hataları, program bug'ları.

- **İnsan hataları:** Yanlış veri girişi, yanlış bant/CD takılması, yanlış programın çalıştırılması, disk/bant kaybı gibi.

ÖNEMLİ

Güvenlik yalnızca "kötü niyetli saldırıya karşı koruma" değildir; **yedekleme (backup)** ve **hata toleransı** da Availability ve Integrity hedeflerinin bir parçasıdır. Kazara veri kaybına karşı en güçlü savunma düzenli ve test edilmiş yedeklemedir.

1.5 İşletim Sisteminin İç Rolü (Internal Roles)

İşletim sistemi, güvenliği destekleyen pek çok **iç özellik** barındırır: **privileged mode** (ayrıcalıklı kip), **memory protection** (bellek koruması), **file access permissions** (dosya erişim izinleri) ve benzeri. Buradaki kritik soru şudur: *Bu özellikler tam olarak neyi başarır ve gerçek hedef nedir?*

Kimi Kime Karşı Koruyoruz? (Protecting Whom?)

İşletim sisteminin koruma mekanizmaları iki yönlü çalışır, ancak her ikisi de tek başına **yeterli değildir**:

- İç özellikler (privileged mode, memory protection) **işletim sistemini kullanıcılara karşı** korur. Bu gereklidir *ama yeterli değildir*.
- Dosya izinleri (**file permissions**) ise **kullanıcıları (ve OS'i) diğer kullanıcılara karşı** korur. Bu da gereklidir *ama yine yeterli değildir*.

DİKKAT — "GEREKLİ AMA YETERLİ DEĞİL"

Slaytlarda iki kez tekrarlanan bu ifade kilit noktadır: OS'in koruma katmanları güvenliğin **temelidir**, ancak güvenliği **garanti etmez**. Çünkü saldırıların çoğu OS korumalarını ihlal etmeden, kullanıcının kendi yetkileri içinde gerçekleşir (ileride "worm"lar ve "It's the Application" bölümünde göreceğiz).

1.6 Kullanıcı Kimlik Doğrulama (User Authentication)

Dosya izinleri **kullanıcı kimliğine (user identity)** dayanır; kullanıcı kimliği ise **kimlik doğrulamaya (authentication)** dayanır. Yani tüm erişim kontrol zinciri, "bu kullanıcı gerçekten iddia ettiği kişi mi?" sorusuna verilen yanıtın üzerine kurulur. İşletim sistemi kullanıcıları üç temel yöntem ailesiyle doğrular:

Kimlik Doğrulamanın Üç Faktörü

Something You Know

Bildiğin bir şey

Parola, PIN,
güvenlik sorusu

Something You Have

Sahip olduğun şey

Token, smart card,
telefon

Something You Are

Olduğun şey

Parmak izi, iris,
biometrics

Şekil 1.2 — Authentication'ın üç faktörü. Birden fazlasını birleştirmek "multi-factor authentication" sağlar.

1.7 Bildiğin Bir Şey: Parolalar (Something You Know: Passwords)

Parolalar en yaygın kimlik doğrulama yöntemidir, ama aynı zamanda en kolay tahmin edilebilen (*easily guessed*) yöntemdir de. Parola güvenliğinin tarihsel gelişimi şöyledir:

- İlk sistemlerde parolalar **düz metin (plaintext)** olarak saklanırdı — bu son derece kötü bir fikirdir, çünkü parola dosyasını ele geçiren herkes tüm parolaları görür.
- Günümüzde parolalar genellikle **hash'lenerek (hashed)** saklanır; yani geri döndürülemeyen bir fonksiyondan geçirilir.
- Ancak **challenge/response** gibi bazı ağ kimlik doğrulama şemaları, doğası gereği düz metin parolayı (veya eşdeğerini) gerektirir.

ÖNEMLİ — BÖLÜM 1 ÖZETİ

- Güvenlik = **Confidentiality + Integrity + Availability** (CIA).
- Tehditler insan kaynaklı (intruders) veya kazara (accidental data loss) olabilir.
- OS'in iç korumaları **gerekli ama yeterli değildir**.
- Erişim kontrol zinciri: **Authentication → User Identity → File Permissions**.
- Kimlik doğrulamanın üç faktörü: bildiğin / sahip olduğun / olduğun şey.

2

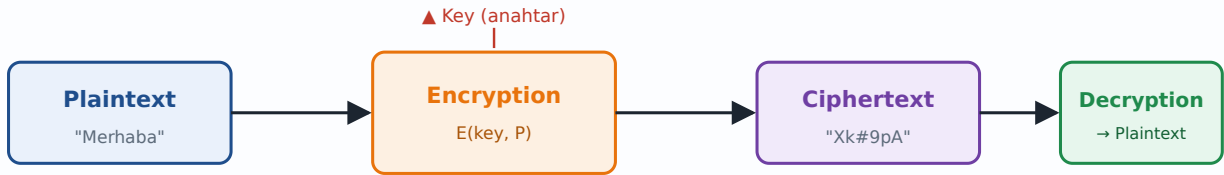
Kriptografinin Temelleri

Secret-Key · Public-Key · Digital Signatures · Hashed Passwords

Kriptografi (**cryptography**), güvenlik hedeflerini matematiksel olarak gerçekleyen araç kutusudur. İşletim sistemi, parolaları korumaktan güvenli iletişime kadar pek çok yerde kriptografiye dayanır. Bu bölümde şifrelemenin temel mantığını, gizli ve açık anahtarlı sistemleri, sayısal imzaları ve parolaların güvenli saklanması inceliyoruz.

2.1 Şifrelemenin Temel Mantığı

Kriptografinin merkezinde **plaintext** (düz/okunabilir metin) ile **ciphertext** (şifreli metin) arasındaki ilişki yer alır. Şifreleme (**encryption**) bir **key** (anahtar) kullanarak plaintext'i ciphertext'e dönüştürür; çözme (**decryption**) ise doğru anahtarla ters işlemi yapar. Temel güvenlik ilkesi şudur: *Algoritma herkesçe bilinse bile, anahtar gizli kaldığı sürece sistem güvenli olmalıdır* (Kerckhoffs ilkesi).



Şekil 2.1 — Plaintext ↔ Ciphertext dönüşümü. Güvenlik anahtarın gizliliğine dayanır, algoritmanın değil.

2.2 Gizli Anahtarlı Kriptografi (Secret-Key Cryptography)

Secret-key (simetrik) kriptografide şifreleme ve çözme aynı gizli anahtarı kullanır. En basit tarihsel örnek **monoalphabetic substitution** (tek-alfabeli yerine koyma) şifresidir: her harf, sabit bir karşılığıyla değiştirilir. Anahtar, 26 harfin permütasyonudur.

ÖRNEK — MONOALPHABETIC SUBSTITUTION

Plaintext : A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Ciphertext: Q W E R T Y U I O P A S D F G H J K L Z X C V B N M

Bu eşlemeyle " CAB " → " EQW " olur. Anahtar uzayı $26! \approx 4 \times 10^{26}$ olsa da, harf frekans analiziyle kolayca kırılır. Bu yüzden modern simetrik şifreler (AES gibi) çok daha karmaşık dönüşümler kullanır.

DİKKAT — SİMETRİK ŞİFRELEMENİN ZAYIF NOKTASI

Secret-key sistemlerin temel sorunu **anahtar dağıtımıdır** (*key distribution*): iki taraf güvenli iletişim kurmadan önce gizli anahtarı güvenli bir kanaldan paylaşmalıdır. Bu sorun, public-key kriptografinin doğuş nedenidir.

2.3 Açık Anahtarlı Kriptografi (Public-Key Cryptography)

Public-key (asimetrik) kriptografi, anahtar dağıtımı sorununu çözer. Her kullanıcının bir **public key** (herkese açık) ve bir **private key** (gizli) çifti vardır. Public key ile şifrelenen veri yalnızca ilgili private key ile çözülebilir. Sistemin güvenliği, tek yönlü (one-way) matematiksel zorluğa dayanır: bazı işlemler **kolayca** yapılırken, anahtarsız tersini almak **hesaplama açısından çok zordur**.

ÖRNEK — "KOLAY" VE "ZOR" İŞLEM ASİMETRİSİ

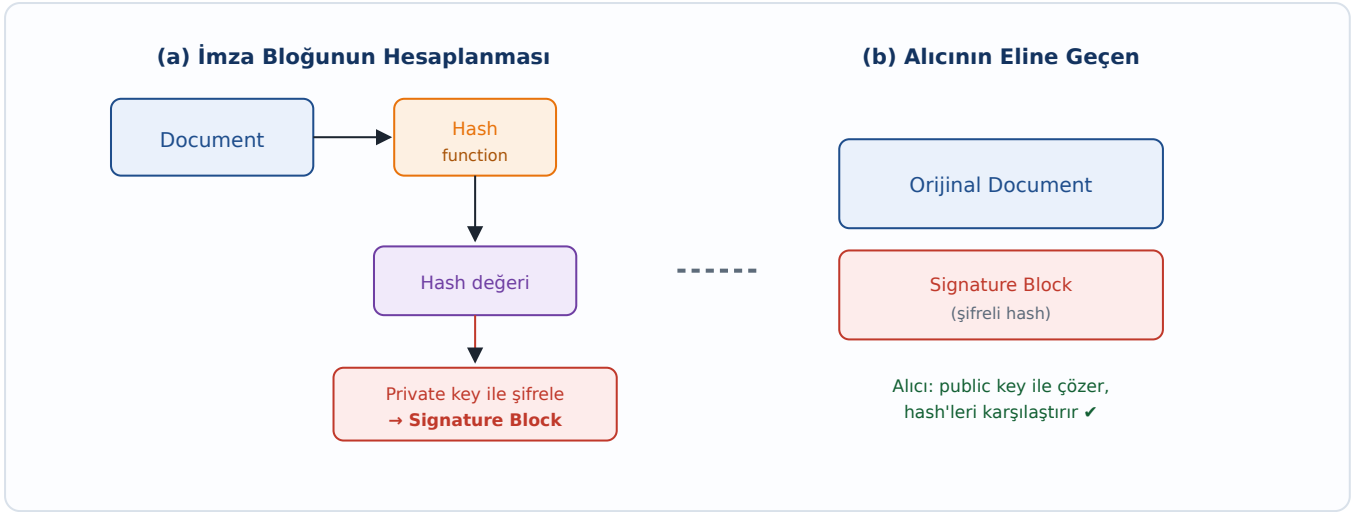
Kolay (şifreleme): $314159265358979 \times 314159265358979 = ? \rightarrow$ Bir çarpma işlemi, saniyeler.

Zor (anahtarsız çözme): $3912571506419387090594828508241$ sayısının karekökü nedir? $\rightarrow$ Büyük sayılarda çarpanlara ayırma çok pahalıdır. RSA tam da bu asimetri (çarpma kolay, çarpanlara ayırma zor) üzerine kuruludur.

Özellik	Secret-Key (Simetrik)	Public-Key (Asimetrik)
Anahtar sayısı	Tek paylaşılan gizli anahtar	Public + Private çift
Hız	Çok hızlı	Yavaş (matematiksel olarak ağır)
Anahtar dağıtımı	Zor (güvenli kanal gerekir)	Kolay (public key serbestçe paylaşılır)
Tipik kullanım	Toplu veri şifreleme (AES)	Anahtar değişimi, digital signature (RSA)

2.4 Sayısal İmzalar (Digital Signatures)

Sayısal imza, bir mesajın **kimden geldiğini** (authentication) ve **değiştirilmediğini** (integrity) kanıtlar. Mantığı public-key kriptografinin tersine işletilmesidir: gönderen, mesajın bir **hash**'ini kendi **private key**'iyle şifreler; bu şifreli hash **signature block**'tur. Alıcı, gönderenin **public key**'iyle imzayı çözer ve mesajın hash'iyle karşılaştırır.



Şekil 2.2 — (a) İmza bloğunun hesaplanması (hash → private key ile şifrele). (b) Alıcı: orijinal belge + signature block alır, public key ile doğrular.

ÖNEMLİ — ŞİFRELEME VS İMZA

Gizlilik için: alıcının *public* key'iyle şifrele → alıcının *private* key'iyle çöz.

İmza için: gönderenin *private* key'iyle imzala → gönderenin *public* key'iyle doğrula. Yani imza, public-key'in **ters yönde** kullanımıdır.

2.5 Hash'lenmiş Parolalar (Hashed Passwords)

Parolaları güvenli saklamanın yolu, parolanın kendisini değil, geri döndürülemez bir **f(PW)** fonksiyonunun çıktısını saklamaktır. Kullanıcı giriş yaptığında sistem girilen parolanın **f(PW)**'sini hesaplar ve saklanan değerle karşılaştırır. Fonksiyon **invertible olmadığı** için, dosyayı ele geçiren saldırgan parolaları doğrudan göremez.

DİKKAT — PRECOMPUTATION ATTACK VE SALT

Saldırgan, yaygın parolaların hash'lerini önceden hesaplayıp bir tabloda saklayabilir (**precomputation / rainbow table attack**). Buna karşı **salt** kullanılır: parola değiştirilirken rastgele bir **s** değeri atanır ve **h(salt, f(PW, salt))** saklanır. Aynı parolaya sahip iki kullanıcı bile farklı salt nedeniyle farklı hash'e sahip olur; böylece önceden hesaplanmış tablolar işe yaramaz.

Salt kullanılsa bile saldırganlar hâlâ **parola tahmin programları** (**password-guessing**) çalıştırabilir. Bu yüzden çoğu işletim sistemi, hash'lenmiş parolaları ayrıca **access control** ile (örneğin yalnızca root'un okuyabileceği bir dosyada) korur — savunma katmanlı (defense in depth) olur.

3

Koruma Mekanizmaları

Protection Domains · Protection Matrix · ACL · Capabilities

İşletim sistemi, "hangi özne (subject) hangi nesne (object) üzerinde hangi işlemi yapabilir?" sorusunu yönetmek için soyut bir model kullanır. Bu bölümde koruma alanları, koruma matrisi ve bu matrisin iki pratik gerçekleştirilmesi olan **Access Control Lists** ile **Capabilities** incelenir.

3.1 Koruma Alanları (Protection Domains)

Bir **protection domain**, <nesne, haklar> (<**object, rights**>) çiftlerinden oluşan bir kümedir. Domain, bir sürecin belirli bir anda erişebileceği nesneleri ve bu nesneler üzerinde sahip olduğu hakları (read, write, execute vb.) tanımlar. Her süreç bir anda bir domain içinde çalışır; UNIX'te (UID, GID) çifti kabaca bir domain belirler.

3.2 Koruma Matrisi (Protection Matrix)

Tüm domainlerin tüm nesneler üzerindeki haklarını tek bir tabloda toplayan soyut yapıya **protection matrix** denir. Satırlar domainleri, sütunlar nesneleri temsil eder; her hücre o domainin o nesne üzerindeki haklarını içerir.

	Object →							
	File1	File2	File3	File4	File5	File6	Printer1	Plot2
Dom 1	Read	Read Write						
Dom 2			Read	Read Write Execute	Read Write		Write	
Dom 3						Read Write Execute	Write	Write

Şekil 3.1 — Bir protection matrix. Satırlar domain, sütunlar object; her hücre o domainin nesne üzerindeki haklarını gösterir.

Domainler de birer nesne gibi düşünülebilir; bu durumda matrise **Domain** sütunları eklenir. Bir domainin başka bir domaine "**Enter**" hakkına sahip olması, o sürecin başka bir domaine **geçebilmesi** anlamına gelir (UNIX'teki **SETUID** bunun bir örneğidir: program çalışırken domain değiştirir).

3.3 Erişim Kontrol Listeleri (Access Control Lists — ACL)

Protection matrix pratikte çok büyük ve seyrek (çoğu hücre boştur). Bu yüzden ya **sütun** ya da **satır** saklanır. **ACL**, matrisi **sütun** bazında saklar: her **nesneye**, o nesneye

erişebilen domain/kullanıcıların ve haklarının bir listesi iliştilir. "Bu dosyaya kim erişebilir?" sorusu doğal olarak yanıtlanır.

ÖRNEK — ACL GÖSTERİMİ

Bir file nesnesinin ACL'i şöyle olabilir: File: { Ali: RW, Veli: R, Grup_Muhasebe: R }. Yani Ali okuyup yazabilir, Veli yalnızca okuyabilir, Muhasebe grubu okuyabilir. UNIX'in `rw-r--r--` izinleri, ACL'in sıkıştırılmış (owner/group/other) bir özel hâlidir.

3.4 Yetkiler (Capabilities)

Capability-based sistemde matris **satır** bazında saklanır: her **sürece** bir **capability list (C-list)** iliştilir. Bu liste, sürecin erişebileceği nesneleri ve her biri üzerindeki haklarını içerir. Capability, taşıyana erişim hakkı veren "anahtar/bilet" gibidir; "bu süreç nelere erişebilir?" sorusu doğal olarak yanıtlanır.

DİKKAT — CAPABILITY'LERİN KORUNMASI

Capability'ler kullanıcı tarafından sahte üretilmemelidir. Çözümlerden biri **cryptographically protected capability**'dir: nesne, haklar ve rastgele bir alanın üzerine bir **check field** (kriptografik hash/MAC) eklenir. Kullanıcı hakları değiştirmeye kalkışırsa check field tutmaz ve capability geçersiz olur.

Genel Haklar (Generic Rights) Örnekleri

- **Copy capability:** Aynı nesne için yeni bir capability oluşturur (hakkı kopyalar).
- **Copy object:** Yeni bir capability ile nesnenin bir kopyasını oluşturur.
- **Remove capability:** C-list'ten bir girdiyi siler; nesne etkilenmez.
- **Destroy object:** Bir nesneyi ve capability'sini kalıcı olarak yok eder.

ACL (sütun = nesne bazlı)

File1 → {A: RW, B: R}
File2 → {B: RWX, C: R}
Printer → {A: W, C: W}

"Bu nesneye kim erişebilir?"

Capabilities (satır = özne bazlı)

Proc A → {File1: RW, Printer: W}
Proc B → {File1: R, File2: RWX}
Proc C → {File2: R, Printer: W}

"Bu özne nelere erişebilir?"

Şekil 3.2 — Aynı protection matrix'in iki gerçekleştirilmesi: ACL nesne-merkezli (sütun), Capabilities özne-merkezli (satır) saklar.

Karşılaştırma	Access Control List (ACL)	Capabilities
Saklama yönü	Sütun (nesne başına liste)	Satır (özne başına liste)
Kolay yanıtlanan soru	"Bu nesneye kim erişebilir?"	"Bu özne nelere erişebilir?"
Hak iptali (revocation)	Kolay — nesnenin listesinden sil	Zor — dağıtılmış capability'leri toplamak gerekir
Hak devri (delegation)	Zor	Kolay — capability'yi ver
Tipik örnek	UNIX/Windows dosya izinleri	Yetenek tabanlı sistemler, token'lar

SINAV İPUCU

Klasik sınav sorusu: "Verilen protection matrix'i (a) ACL ve (b) capability list olarak gösteriniz." Yöntemi hatırlayın: **ACL = sütunları oku** (her nesne için kimlerin hangi hakkı var), **Capability = satırları oku** (her özne için hangi nesnede hangi hak var). Ayrıca revocation'ın ACL'de neden kolay, capability'de neden zor olduğunu açıklayabilmelisiniz.

4

Güvenilir Sistemler ve Formal Modeller

Trusted Computing Base · Reference Monitor · Bell-La Padula · Biba

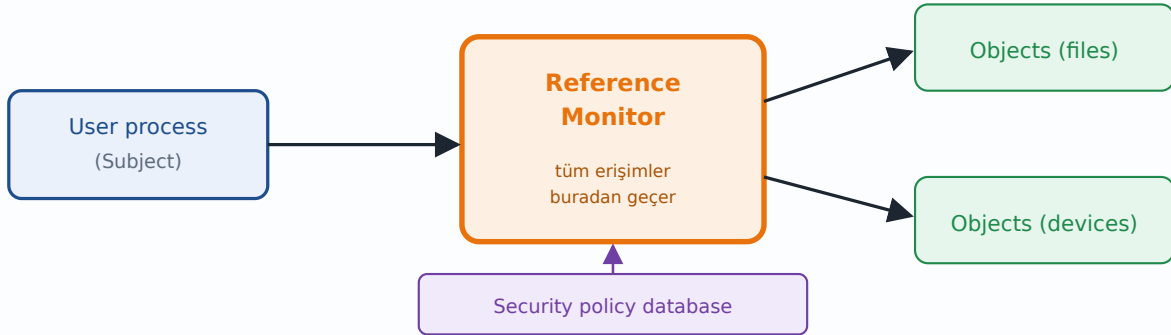
Virüsler, solucanlar ve sürekli güvenlik açıkları karşısında iki masum ama derin soru sorulur: *Güvenli bir bilgisayar sistemi inşa etmek mümkün müdür? Mümkünse neden yapılmıyor?* Bu bölüm, güvenliği matematiksel olarak **kanıtlanabilir** kılma çabasını — güvenilir hesaplama tabanını ve formal güvenlik modellerini — ele alır.

4.1 Güvenilir Sistemler (Trusted Systems)

"Güvenli sistem inşa edilebilir mi?" sorusunun yanıtı teorik olarak **evet**'tir; sorun bunun pratik ve ticari olarak **maliyetli, kullanışsız ve eskiye göre geri kalmış** olmasıdır (Bölüm 10'da Orange Book bağlamında detaylandıracağız). Güvenilir bir sistemin temelinde, güvenlik politikasını zorlayan ve güvenilmesi gereken çekirdek bir bileşen yatar.

4.2 Güvenilir Hesaplama Tabanı (Trusted Computing Base — TCB)

TCB, sistemin güvenliğini sağlamaktan sorumlu olan ve doğruluğuna **güvenmek zorunda olduğumuz** donanım+yazılım bileşenlerinin toplamıdır. TCB'nin kalbinde **reference monitor** (referans gözlemci) bulunur: tüm güvenlikle ilgili erişimleri (özne → nesne) **tek bir noktadan** geçiren ve politikaya göre izin veren/reddeden mekanizma.



Şekil 4.1 — Reference monitor: özne ile nesne arasındaki tüm erişimler tek noktadan geçer ve güvenlik politikasına göre denetlenir.

ÖNEMLİ — İYİ BİR REFERENCE MONİTOR'ÜN 3 ÖZELLİĞİ

- **Complete mediation:** Hiçbir erişim atlanamaz; her erişim denetlenir.
- **Tamper-proof:** Kurcalanamaz, değiştirilemez.
- **Verifiable / küçük:** Doğruluğu analiz edilebilecek kadar küçük ve basit olmalı.

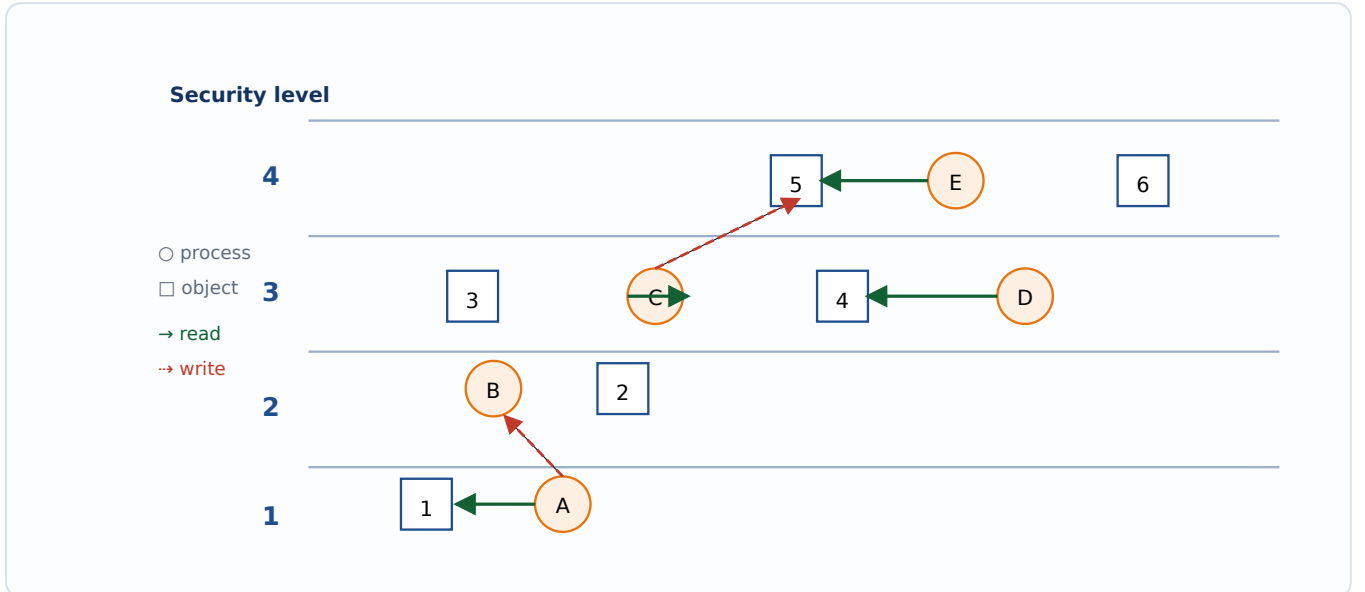
4.3 Güvenli Sistemlerin Formal Modelleri

Güvenliği formal kılmak için sistem, **durumlar (states)** kümesi olarak modellenir. Her durum **authorized (yetkili/güvenli)** ya da **unauthorized (yetkisiz)** olabilir. Bir sistem **güvenlidir** ancak ve ancak güvenli bir durumdan başlayıp uygulanabilir komutlarla **hiçbir zaman unauthorized bir duruma geçemiyorsa**. Bu, güvenliği bir matematiksel ispat problemine dönüştürür.

4.4 Bell-La Padula Modeli (Gizlilik için)

Bell-La Padula (BLP), çok seviyeli güvenlik (**multilevel security**) için klasik bir modeldir ve amacı **gizliliği (confidentiality)** korumaktır. Süreçler ve nesneler güvenlik seviyelerine atanır. İki kural vardır:

- **Simple security property (no read up):** Seviye k 'deki bir süreç, yalnızca **kendi seviyesindeki veya daha düşük** nesneleri **okuyabilir**. Örnek: bir general, bir binbaşının belgesini okuyabilir; tersi olmaz.
- *** (star) property (no write down):** Seviye k 'deki bir süreç, yalnızca **kendi seviyesindeki veya daha yüksek** nesnelere **yazabilir**. Örnek: bir binbaşı, generalin posta kutusuna bildiklerini ekleyebilir; ama generalin aynı şeyi düşük seviyeye yazması gizliliği ihlal eder.



Şekil 4.2 — Bell-La Padula çok seviyeli güvenlik modeli. Düz ok = read (aşağı/eşit), kesik ok = write (yukarı/eşit). "No read up, no write down".

DİKKAT — BLP YALNIZCA GİZLİLİĞİ KORUR

Bell-La Padula "sırları saklamak" (**keep secrets**) için tasarlanmıştır. Peki ya **veri bütünlüğü (data integrity)**? BLP'de düşük seviyeli bir süreç yukarı yazabildiği için, kötü niyetli düşük-seviye bir aktör yüksek-seviye veriyi **bozabilir**. Bütünlük sorununu Biba modeli çözer.

4.5 Biba Modeli (Bütünlük için)

Biba modeli, BLP'nin **tam tersi** kuralları uygulayarak **integrity**'yi korur. Mantık şudur: yüksek bütünlük seviyesindeki veriler, düşük bütünlükteki ("kirli") kaynaklardan etkilenmemelidir.

- **Simple integrity principle (no write up):** Seviye k 'deki süreç yalnızca **kendi seviyesindeki veya daha düşük** nesnelere **yazabilir**.
- **Integrity * property (no read down):** Seviye k 'deki süreç yalnızca **kendi seviyesindeki veya daha yüksek** nesneleri **okuyabilir**.

Model	Korur	Read kuralı	Write kuralı
Bell-La Padula	Confidentiality (gizlilik)	No read up (yukarı okuma yok)	No write down (aşağı yazma yok)
Biba	Integrity (bütünlük)	No read down (aşağı okuma yok)	No write up (yukarı yazma yok)

4.6 İkisi Birden? (How about both?)

DİKKAT — BLP + BIBA ÇATIŞMASI

Hem Bell-La Padula hem de Biba özelliklerini **aynı anda** istersek bir çelişki doğar: BLP "no read up + no write down", Biba ise "no read down + no write up" der. Bu kuralların kesişimi, bir sürecin yalnızca **tam kendi seviyesindeki** nesnelerle (read+write) etkileşmesine izin verir — yani seviyeler arası akış neredeyse tamamen kilitlenir. Bu, gizlilik ve bütünlüğün aynı modelde birleştirilmesinin neden zor olduğunu gösterir.

5

Covert Channels (Gizli Kanallar)

Confinement Problem · Bilgi Sızdırma · Steganography

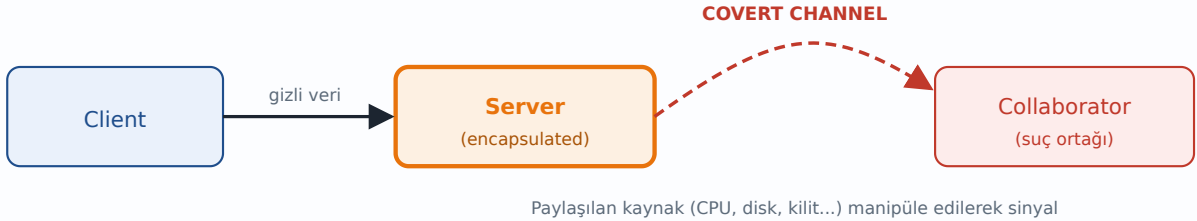
Erişim kontrolü mükemmel olsa bile, kötü niyetli bir program bilgiyi **tasarlanmamış kanallar** üzerinden dışarı sızdırabilir. Bu, güvenlik modellerinin temel bir sınırını ortaya koyar: **confinement problem**.

5.1 Confinement Problem ve Covert Channel Tanımı

Senaryo: bir **client**, gizli verisini işlemesi için bir **server**'a verir. Server'a güvenmek istemeyiz; bu yüzden onu izole eder (encapsulate) ederiz. Ancak server'ın gizli veriyi bir **collaborator** (suç ortağı) sürece sızdırmasını tamamen engelleyebilir miyiz? Buna **confinement problem** denir.

TANIM — COVERT CHANNEL

Bir **covert channel**, iletişim için **tasarlanmamış** bir mekanizmayı kötüye kullanarak bilgi aktaran gizli kanaldır. İzole edilmiş bir server, doğrudan iletişim kuramasa bile, paylaşılan sistem kaynaklarını manipüle ederek collaborator'a sinyal gönderebilir.



Şekil 5.1 — Encapsulate edilmiş server, doğrudan iletişim kuramasa da covert channel üzerinden collaborator'a bilgi sızdırabilir.

5.2 Örnek: Dosya Kilidiyle Covert Channel

Klasik bir covert channel örneği **file locking** kullanır. Server ve collaborator, önceden anlaşmış bir dosyaya bakar. Server, sızdırmak istediği her bit için: **1 göndermek için** dosyayı kilitler, **0 göndermek için** kilitlemez. Collaborator dosyanın kilitli olup olmadığını periyodik kontrol ederek bit dizisini okur. Hiçbir "iletişim" izni ihlal edilmeden, paylaşılan kilit durumu bir iletişim kanalına dönüşür.

5.3 Steganography (Veri Gizleme)

Bilgi, masum görünen verinin içine de gizlenebilir; buna **steganography** denir. Tanenbaum'un çarpıcı örneği: bir **zebra fotoğrafı** ile, içine Shakespeare'in beş oyununun tüm metni gizlenmiş görünüşte aynı zebra fotoğrafı yan yana konduğunda gözle ayırt edilemez. Görüntünün piksellerinin en düşük anlamlı bitleri (LSB) değiştirilerek büyük miktarda veri, görüntü kalitesini gözle fark edilir biçimde bozmadan saklanabilir.

ÖNEMLİ — COVERT CHANNEL VS STEGANOGRAPHY

Covert channel: İki süreç arasında, paylaşılan kaynağı zamansal/durumsal manipüle ederek kurulan gizli *iletişim kanalı*.

Steganography: Verinin başka bir verinin (resim, ses) içine, varlığı gizlenecek şekilde *gömülmesi*. Covert channel "nasıl iletilir", steganography "nereye saklanır" sorusuna odaklanır.

6

Kimlik Doğrulama (Authentication)

Passwords · Challenge/Response · Tokens · Smart Cards · Biometrics

Tüm erişim kontrolü, kullanıcının kimliğinin doğru tespitine dayanır. Bu bölüm, kimlik doğrulamanın üç faktörünü — bildiğin, sahip olduğun, olduğun şey — ayrıntılı olarak işler ve gerçek saldırı senaryolarını inceler.

6.1 Genel İlkeler

Kullanıcı doğrulamanın üç genel ilkesi şunlardır:

- **Something the user knows** — bildiği bir şey (parola, PIN).
- **Something the user has** — sahip olduğu bir şey (token, smart card).
- **Something the user is** — olduğu bir şey (biometrics: parmak izi, iris).

6.2 Parolalarla Kimlik Doğrulama (Authentication Using Passwords)

Bir login işleminde sistem kullanıcı adı ve parola ister. Güvenlik açısından önemli bir tasarım kararı, **hatanın ne zaman bildirildiğidir**. İyi tasarlanmış bir sistem, kullanıcı adı geçersiz olsa bile **parola da girilene kadar** reddetmez — böylece saldırgan, hangi kullanıcı adlarının geçerli olduğunu giriş ekranından öğrenemez.

DİKKAT — BİLGİ SIZINTISI (INFORMATION LEAKAGE)

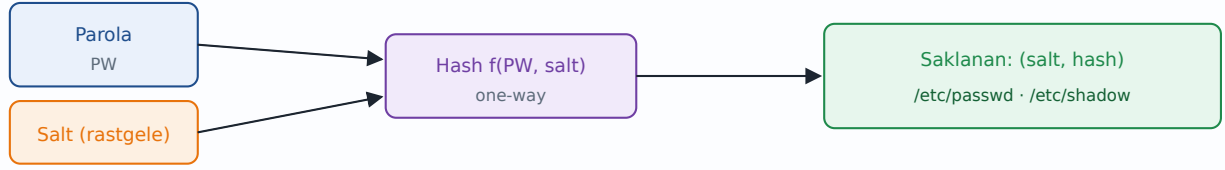
Slaytlardaki üç senaryo: (a) başarılı login, (b) kullanıcı adı girildikten **hemen sonra** ret, (c) ad **ve** parola girildikten sonra ret. (b) seçeneği kötüdür: "bu kullanıcı adı yok" bilgisini sızdırır. Doğru tasarım (c)'dir: her durumda aynı genel "login incorrect" mesajını ver ve aynı süreyi harca (timing attack'ı önlemek için).

6.3 Crackerlar Nasıl İçeri Girer? (How Crackers Break In)

Tanenbaum, bir cracker'ın LBL'deki (Lawrence Berkeley Lab) bir ABD Enerji Bakanlığı bilgisayarına nasıl sızdığını örnek verir. Tipik bir saldırı, zayıf/varsayılan parolaların ve bilinen güvenlik açıklarının sistematik denenmesiyle ilerler — saldırgan bir makineden diğerine sıçrayarak (lateral movement) ağ içinde yayılır.

6.4 UNIX Parola Güvenliği ve Salt

UNIX'in klasik parola koruması, Bölüm 2'de tanıtılan **salt** mekanizmasının somut bir uygulamasıdır. Kullanıcının parolası, kendisine atanmış rastgele bir salt ile birlikte hash'lenir ve hem salt hem de sonuç saklanır. Bu, **precomputation** (önceden hesaplama) saldırılarını etkisiz kılar.



Şekil 6.1 — Salt'ın precomputation'ı engellemesi: parola, rastgele salt ile birlikte hash'lenir; (salt, hash) çifti saklanır.

6.5 Challenge/Response Kimlik Doğrulama

Challenge/response şemasında parola ağ üzerinden hiç gönderilmez. Sunucu kullanıcının parolasını (PW) bilir ve rastgele bir sayı **N** (challenge) gönderir. Her iki taraf da **f(PW, N)** değerini hesaplar (f bir şifreleme/hash fonksiyonudur); kullanıcı sonucu (response) geri yollar. Sunucu kendi hesabıyla karşılaştırır.

DİKKAT — CHALLENGE/RESPONSE'UN SINIRI

N ve f(PW, N)'i gözlemleyen bir saldırgan (eavesdropper), hâlâ **offline parola tahmini** (password-guessing) yapabilir: tahmin ettiği her PW' için f(PW', N) hesaplar ve gözlemlediği değerle karşılaştırır. Bu nedenle güçlü parola hâlâ kritiktir. Ayrıca bu şema, sunucunun parolayı düz metin (veya eşdeğeri) saklamasını gerektirir.

ÖRNEK — İNSANIN HATIRLAYABİLECEĞİ SORULAR

Kullanıcının yazmak zorunda kalmayacağı (akılda tutulabilen) sorular seçilmelidir: "Marjolein'in kız kardeşi kim?", "İlkokulun hangi caddede idi?", "Mrs. Woroboff hangi dersi verirdi?". **Uyarı:** annenizin kızlık soyadı uygun değildir — kamuya açık kayıtlardan bulunabilir.

6.6 İnsan Faktörü (The Human Element)

ÖNEMLİ — ÜNLÜ ALINTI (NETWORK SECURITY KİTABINDAN)

"İnsanlar yüksek kaliteli kriptografik anahtarları güvenli saklamada yetersizdir ve kriptografik işlemleri kabul edilemez bir hız ve doğrulukta yaparlar. Ayrıca büyük, bakımı pahalı, yönetimi zor varlıklardır... Bu cihazların hâlâ üretilip konuşlandırılması şaşırtıcıdır, ama o kadar yaygınlar ki protokollerimizi onların kısıtlarına göre tasarlamak zorundayız."

Esprili yollu bu alıntı kritik bir gerçeği vurgular: güvenlik sistemlerinin en zayıf halkası genellikle **insandır**. Bu yüzden sosyal mühendislik (social engineering) çoğu zaman teknik saldırılardan daha etkilidir.

6.7 Sahip Olduğın Şey: Token'lar (Something You Have)

Token, genellikle kurcalamaya dayanıklı (**tamper-resistant**) bir cihazdır. Token, challenge/response'u kendisi yapabilir. Çok yaygın bir tür, içinde bir **kriptografik sır (K)** barındırır ve günün saatini (T) kullanarak **f(K, T)** hesaplar — ekrandaki sürekli değişen kod budur. Kayıp/çalınmaya karşı genellikle bir **PIN** ile birleştirilir (iki faktör).

Smart card ile kimlik doğrulamada kart, içindeki sırrı dışarı çıkarmadan challenge/response hesaplamasını kendi üzerinde yapar — sır kartın dışına asla çıkmaz.

6.8 Olduğın Şey: Biyometrik (Something You Are: Biometrics)

Biyometrik doğrulama, kullanıcının fiziksel özelliklerine dayanır: parmak izi okuyucular yaygınlaşmıştır, iris taramaları ise muhtemelen daha güvenlidir. Tanenbaum'un örnek cihazı bir **parmak uzunluğu ölçüm** aletidir.

DİKKAT — BİYOMETRİĞİN KRİTİK SORUNLARI

- **False positive / false negative oranları:** Hiçbir biyometrik sistem mükemmel değildir. Parametreleri çok gevşek ayarlarsanız yetkisiz kişileri kabul edersiniz (false positive); çok sıkı ayarlarsanız yetkili kişileri reddedersiniz (false negative). Bu denge dikkatle kurulmalıdır.
- **Yerel saklama:** Biyometrik en iyi **yerelde** saklandığında çalışır. Ağ üzerinden gönderilince görülen şey yalnızca bir **bit dizisidir** ve bu dizi tekrar oynatılabilir.
- **Spoofing (taklit):** Sahte parmak izi, fotoğrafla iris taklidi gibi saldırılara dikkat edin.
- **Geri alınamazlık:** Parola değiştirilebilir; ama parmak izinizi "değiştiremezsiniz". Bir kez sızdırılırsa kalıcı olarak risktedir.

SINAV İPUCU

"Üç authentication faktörünü örneklerle açıklayın ve her birinin zayıflıklarını tartışın" tarzı bir karşılaştırmalı soru beklenebilir. Anahtar zayıflıklar: parola → tahmin/çalınma; token → kayıp/çalınma; biyometri → spoofing + geri alınamazlık + false positive/negative.

Bu bölüm, saldırganların sistemlere girmek için kullandıkları somut teknikleri inceler. Üç ana başlık öne çıkar: **Trojan horses**, **login spoofing** ve hepsinin en büyüğü — **buggy software** (hatalı yazılım), özellikle buffer overflow sınıfı açıklar.

7.1 Genel Saldırı Sınıfları

- **Trojan horses** — "gel de al" (come and get it) tarzı, kullanıcıyı kandırma saldırısı.
- **Login spoofing** — sahte giriş ekranıyla kimlik bilgisi çalma.
- **Buggy software** — **en büyük tehdit**; yazılımdaki hataların sömürülmesi.

7.2 Truva Atları (Trojan Horses)

Trojan horse, kullanıcıyı kötücül işler yapan bir programı çalıştırmaya kandırır (çoğu virüs ve solucan bu yolla yayılır). Peki OS kullanıcıyı nasıl korur? Sorun şudur: UNIX tarzı dosya izinleri **yardımcı olmaz**, çünkü saldırı programı kullanıcının kendi yetkileriyle çalışır ve dosya izinlerini bizzat değiştirebilir. Çözüm **mandatory access control (MAC)** gerektirir — kullanıcının bile değiştiremediği, sistem tarafından zorunlu kılınan politikalar.

TANIM — DAC VS MAC

DAC (Discretionary Access Control): Erişim kararını **nesnenin sahibi** verir; izinler sahibinin takdirindedir (UNIX dosya izinleri). Trojan, sahibin yetkisiyle çalışıp izinleri değiştirebilir.

MAC (Mandatory Access Control): Erişim kararını **sistem politikası** zorunlu kılar; kullanıcı bunu gevşetemez. Bell-La Padula gibi modeller MAC ile uygulanır.

7.3 Tuzak Kapıları (Trap Doors)

Trap door (backdoor), normal kimlik doğrulamayı atlayan, programa **gizlice yerleştirilmiş** bir koddur. Örneğin login kodunun içine "kullanıcı adı `zzzzz` ise parolasız giriş ver" şeklinde bir kontrol eklenmesi. Normal kodla trap door'lu kod neredeyse aynı görünür; farkı yakalamak code review gerektirir.

(a) Normal kod

```
while (TRUE) {  
    printf("login: ");  
    get_string(name);  
    v = check(name, pw);  
    if (v) break;  
}
```

(b) Trap door eklenmiş

```
while (TRUE) {  
    printf("login: ");  
    get_string(name);  
    if (eq(name, "zzzzz")) break;  
    v = check(name, pw);  
    if (v) break; }
```

Şekil 7.1 — (a) Normal login döngüsü. (b) Tek satırlık bir trap door: özel kullanıcı adı doğrulamayı atlatır.

7.4 Sandbox'lar (Sandboxes)

OS'in daha iyi yaklaşımı, programlara bir **sandbox** sunmaktır: programın çalışabildiği ama makinenin geri kalanını **etkileyemediği** bir ortam. Güçlü izolasyon kavramsal olarak kolaydır — programı ayrı bir makinede veya VMware gibi bir sanal makinede çalıştırın. Daha zarif (düşük maliyetli) mekanizmalar da vardır. Asıl zorluk şudur: programın dış dünyayla **sınırlı etkileşimine** izin verirken güvenliği korumak. (Detaylı sandboxing tekniklerini Bölüm 9'da göreceğiz.)

7.5 Login Spoofing

Login spoofing'de saldırgan, gerçeğine birebir benzeyen **sahte bir login ekranı** çalıştırır. Kullanıcı adını ve parolasını girince bunlar saldırgana kaydedilir, ardından genellikle "login incorrect" gösterilip gerçek ekrana yönlendirilir — kullanıcı hiçbir şey fark etmez. (Bugün sahte ATM'lerde benzer şeyleri görüyoruz.)

Trusted Path (Güvenilir Yol) Çözümü

Çözüm **trusted path**'tir: kullanıcının başlattığı ve **kesinlikle gerçek OS'e** ulaşmayı garanti eden bir tuş dizisi. Klasik örnek Windows'taki **Ctrl+Alt+Delete**'tir (Secure Attention Sequence). Bu kombinasyon yalnızca çekirdek tarafından yakalanabildiği için, hiçbir kullanıcı programı onu taklit edemez. Ancak insanların eğitilmesi gerekir: "Bu diziyi **siz başlatmadıkça** asla giriş yapmayın." Tüm parola istemleri bu şekilde korunmalıdır.

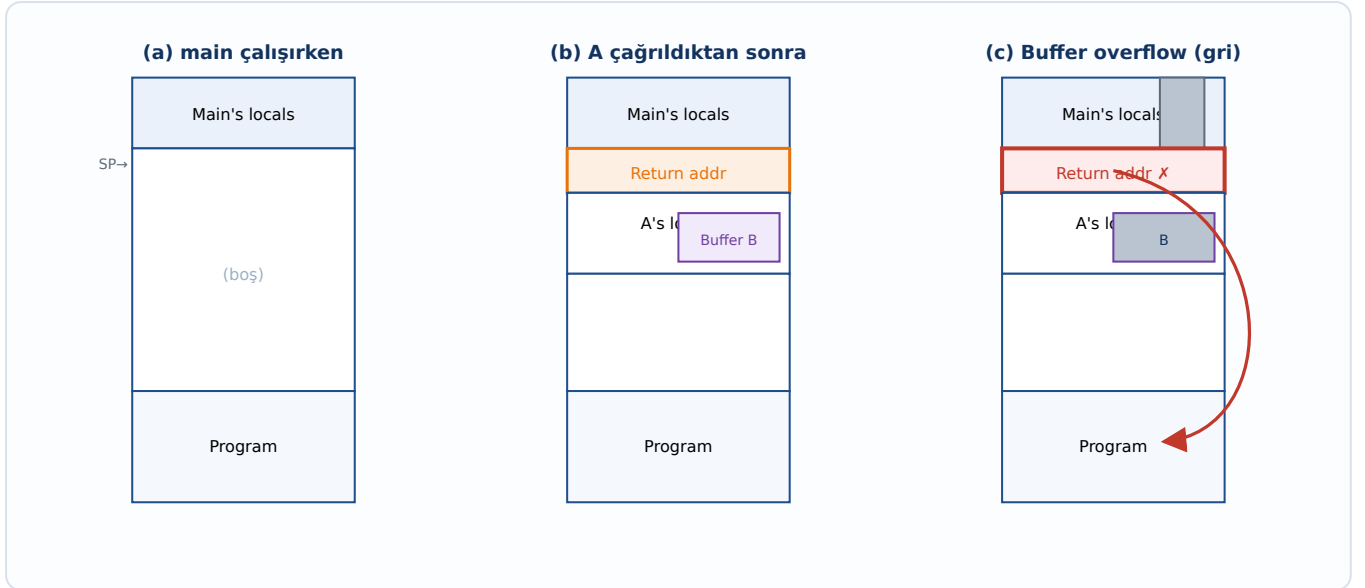
7.6 Kod Hatalarını Sömürme (Exploiting Code Bugs)

Bir bug'ı sömürmenin tipik adımları:

- telnet bağlantısı kabul eden makineleri bulmak için **port scan** yap.
- login adı/parola kombinasyonlarını tahmin ederek girmeye çalış.
- İçeri girince, hatalı programı, bug'ı tetikleyen bir girdiyle çalıştır.
- Hatalı program **SETUID root** ise, bir SETUID root shell oluştur.
- Bir IP portunu dinleyip komut bekleyen bir **zombie** programı indir ve başlat.
- Zombie'nin sistem yeniden başladığında otomatik çalışmasını ayarla (kalıcılık/persistence).

7.7 Buffer Overflow Saldırıları

Buffer overflow, güvenlik tarihinin en önemli açık sınıfıdır. C dili, dizi sınırlarını otomatik denetlemez. Bir prosedür çağrıldığında **return address** (geri dönüş adresi) ve yerel değişkenler yığına (stack) yerleştirilir. Eğer bir yerel **buffer**'a, kapasitesinden fazla veri yazılırsa, taşan veri komşu bellek hücrelerini — kritik olarak **return address**'i — ezer. Saldırgan bu adresi kendi yerleştirdiği kötücül koda (shellcode) yönlendirerek programın kontrolünü ele geçirir.



Şekil 7.2 — Buffer overflow. (a) main çalışıyor. (b) Prosedür A çağrılınca return addr + Buffer B yığına konur. (c) B'ye taşan veri (gri) return addr'i ezer; program saldırganın koduna döner.

DİKKAT — NEDEN STACK YUKARI DOĞRU TAŞAR?

Yığın yüksek adreslerden düşük adreslere doğru büyür, ama bir buffer'a yazma **düşükten yükseğe** doğru ilerler. Bu yüzden buffer'dan taşan veri, daha **yüksek adreslerdeki** return address'e doğru ilerleyerek onu ezer. Bu, return address'i ele geçirmenin temel mekanizmasıdır.

7.8 Return-to-libc Saldırıları

Modern savunmalar (örneğin **DEP / NX-bit** — yığını "non-executable" yapma) saldırganın yığına kod koyup çalıştırmasını engeller. **Return-to-libc** bunu aşar: saldırgan kendi kodunu enjekte etmek yerine, return address'i sistemde **zaten var olan** bir kütüphane fonksiyonuna (örneğin `system()`) yönlendirir ve yığını uygun argümanlarla (örneğin `"/bin/sh"`) doldurur. Böylece yeni kod çalıştırmaya gerek kalmadan mevcut güvenilir kod kötüye kullanılır. (Bu fikrin genellemesi **ROP — Return-Oriented Programming**'dir.)

7.9 Code Injection Saldırıları

Code injection, kullanıcı girdisinin yeterince temizlenmeden (sanitize edilmeden) bir komut/sorgu içine yerleştirilmesinden doğar. Klasik örnek, bir programın kullanıcıdan aldığı

dosya adını doğrudan bir sistem komutuna (örneğin `system("rm " + filename)`) gömmesidir. Kullanıcı dosya adı yerine `x; rm -rf /` girerse, kabuk bunu iki ayrı komut olarak çalıştırır ve saldırgan keyfi komut yürütür. (Web dünyasındaki **SQL injection** ve **command injection** aynı kök nedene dayanır.)

ÖNEMLİ — BUFFER OVERFLOW VS CODE INJECTION

Buffer overflow: Bellek sınırı denetlenmediği için return address/bellek **ezilir**; kontrol akışı ele geçirilir.

Code injection: Girdi **temizlenmediği** için, veri olması gereken girdi **kod/komut** olarak yorumlanır. Ortak ders: **her girdiye düşman gözüyle bak** (input validation).

7.10 Heartbleed (Gerçek Dünya Vaka İncelemesi)

Heartbleed (2014), OpenSSL'deki ünlü bir açıktır ve "uzunluk hakkında yalan söyleme" sınıfının tipik örneğidir. HTTPS bağlantısında, oturumu canlı tutmak için bir **heartbeat** paketi gönderilir; bu paket, bir veri parçası ve o verinin **uzunluğunu bildiren bir alan** içerir.

- Saldırı, heartbeat paketinde **uzunluk alanını gerçeğinden büyük** (örneğin maksimum 64 KB) bildirir, ama aslında çok küçük veri gönderir.
- Hatalı sunucu, bildirilen uzunluk kadar veriyi **bellekten** kopyalayıp yanıt paketine koyar.
- O bellekte ne olduğu belirsizdir: **şifreleme anahtarları, oturum bilgileri, login bilgileri** sızabilir.
- Çözüm basittir: gönderenin bildirdiği uzunluğun gerçek veri uzunluğuyla tutarlı olup olmadığını **kontrol etmek** — yani basit bir **bounds check**.

DİKKAT — HEARTBLEED'İN İZ BIRAKMAMASI

Bu açığın sömürülmesi sunucuda **hiçbir iz bırakmaz**; bu yüzden ne kadar "güvenli" verinin çalındığını ölçmek imkânsızdır. BAE Security verilerine göre en çok ziyaret edilen 10.000 sitenin 628'i 8 Nisan'da savunmasızken, sitelerin hızlı yama uygulamasıyla bu sayı günler içinde 180'e düştü. Ders: güvenlik açıkları yalnızca teknik değil, **operasyonel yama hızı** (agility) meselesidir de.

8

Kötücül Yazılımlar (Malware)

Viruses · Worms · Ransomware · Spyware · Rootkits

Bu bölüm, kötücül yazılım (malware) ekosistemini ele alır: virüsler, solucanlar, fidye yazılımları, casus yazılımlar ve sistemin en derinine gizlenen rootkit'ler. Her birinin yayılma ve gizlenme stratejisi farklıdır.

8.1 Virüsler ve Solucanlar (Viruses and Worms)

TANIM — VIRUS VS WORM

Virus (Virüs): Bir makine **içinde** kendi kendine yayılır (başka dosyalara bulaşır), ancak başka makineye bulaşmak için **insan müdahalesi gerektirir** (örneğin enfekte dosyanın kopyalanması).

Worm (Solucan): Makineler **arasında** kendi kendine yayılır; bazen insan yardımı (eki açma) gerekse de ağ üzerinden otonom yayılma yeteneği vardır.

Kilit soru: OS bunları durdurmak için ne yapabilir? Yanıt sınırlıdır — çünkü çoğu solucan, OS güvenliğini ihlal etmeden, kullanıcının kendi yetkileriyle yayılır (Bölüm 10'da bu konuya döneceğiz).

8.2 Çalıştırılabilirleri Engelleme (Blocking Executables)

OS, şüpheli içeriği engellemeye çalışabilir, ama bunu yapmak **çok zordur** — kötücül kodu sokmanın sayısız yolu vardır. Örneğin Windows XP SP2, indirilen dosyaları "etiketler" (tag) ve etiketli her şeyi çalıştıramaz sayar. Ancak bu yaklaşımın temel ikilemi şudur: indirilmesine **izin vermeniz gereken** bug fix'ler / meşru güncellemeler ne olacak? İyi ile kötüyü ayırmak otomatik olarak mümkün değildir.

8.3 Fidye Yazılımı (Ransomware) ve Genel Malware

Malware bir **şantaj** aracı olarak kullanılabilir. Tipik **ransomware** senaryosu: kurbanın diskindeki dosyaları şifreler ve bir mesaj gösterir — "Diskinizin şifre çözme anahtarını satın almak için işaretsiz küçük banknotlarla 100 dolar gönderin..." gibi. Şifreleme güçlü olduğunda, anahtarsız veri kurtarmak pratikte imkânsızdır; bu yüzden ransomware'e karşı en güçlü savunma yine **çevrimdışı yedeklemedir**.

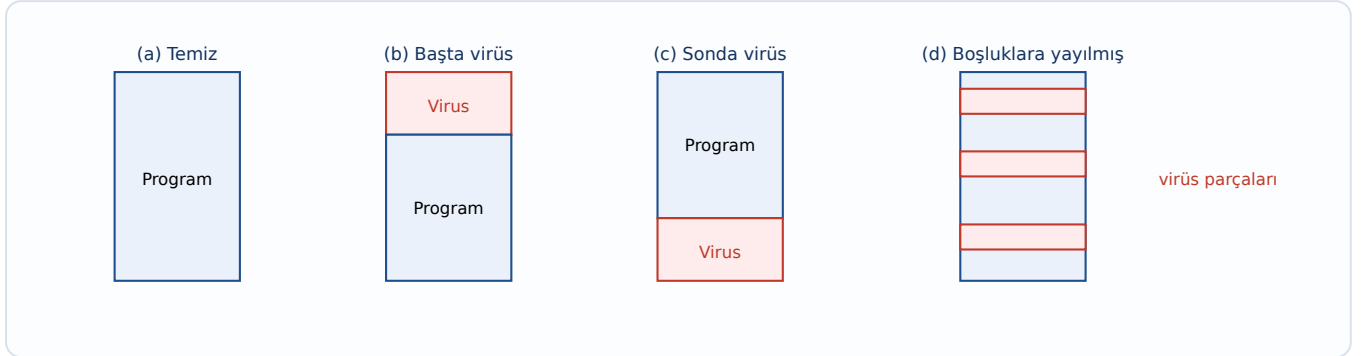
8.4 Virüs Türleri (Types of Viruses)

Virüs Türü	Çalışma Mantığı
Companion virus	Programı değiştirmez; aynı adla farklı uzantıda (örn. .com vs .exe) bir eşlik dosyası oluşturup önce kendi çalışır.
Executable program virus	Çalıştırılabilir dosyaların içine yerleşir; program her çalıştığında virüs de çalışır.
Parasitic virus	Kendini var olan bir programa ilştirir (başına, sonuna veya boşluklara).
Memory-resident virus	Bellekte kalıcı olur, sistem çağrılarını yakalayarak sürekli aktif kalır.
Boot sector virus	Disk boot sektörüne yerleşir; OS yüklenmeden önce çalışır.
Device driver virus	Bir aygıt sürücüsüne bulaşır; sürücü yüklenince yüksek yetkiyle çalışır.
Macro virus	Office belgelerindeki makrolara gömülür (örn. Word/Excel makroları).
Source code virus	Kaynak kod dosyalarına bulaşır; derleme sırasında yayılır.

Executable Program Virüsleri ve Bulaşma

Bir executable virüs, sisteme yayılmak için çalıştırılabilir dosyaları bulmak ister. Tanenbaum'un örneği, bir UNIX sisteminde dizinleri **özyinelemeli (recursive)** dolaşp çalıştırılabilir dosyaları bulan bir prosedürdür. Virüs, bulduğu her uygun dosyaya kendini ekleyerek yayılır.

Parasitic Virüslerin Yerleşim Stratejileri



Şekil 8.1 — Parasitic virüsün yerleşim yolları: (a) temiz program, (b) başına, (c) sonuna, (d) programın içindeki boş alanlara dağılmış.

Boot Sector Virüsleri

Boot sector virüsü, OS yüklenmeden çalıştığı için interrupt ve trap vektörlerini ele geçirebilir. OS yüklenince bazı vektörleri geri alır; virüs vektör kaybını fark edip yeniden ele geçirir — böylece OS ile bir "vektör kapma" mücadelesine girer. Boot sektörüne yerleşmek, virüse OS'ten **daha erken ve daha ayrıcalıklı** çalışma fırsatı verir.

8.5 Casus Yazılım (Spyware)

Spyware, sahibinin bilgisi olmadan gizlice bir PC'ye yüklenen ve arka planda kullanıcının aleyhine işler yapan yazılımdır. Dört temel karakteristiği:

- **Gizlenir:** Kurban onu kolayca bulamaz.
- **Veri toplar:** Kullanıcı hakkında bilgi (gezinme, tuş vuruşları vb.) biriktirir.
- **Geri iletir:** Topladığı bilgiyi uzaktaki "efendisine" (distant master) gönderir.
- **Hayatta kalır:** Kaldırılma girişimlerine direnir (persistence).

Spyware Nasıl Yayılır?

Malware/Trojan ile aynı yollar; **drive-by download** (enfekte web sitesini ziyaret etmek), web sayfasının bir `.exe` çalıştırmaya çalışması, kullanıcının farkında olmadan enfekte bir toolbar kurması, kötücül ActiveX kontrollerinin yüklenmesi.

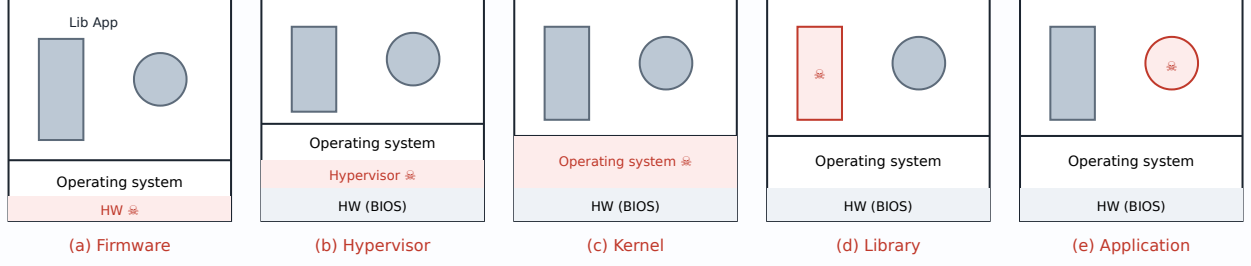
Spyware'in Eylemleri

Tarayıcının ana sayfasını/yer imlerini değiştirme, yeni toolbar ekleme, varsayılan medya oynatıcı ve arama motorunu değiştirme, masaüstüne ikon ekleme, web reklamlarını kendi seçtikleriyle değiştirme, Windows diyaloglarına reklam koyma ve durdurulamaz pop-up reklam akışı üretme gibi.

8.6 Rootkit'ler (Rootkits)

Rootkit, varlığını ve diğer malware'in varlığını **gizlemek** için sistemin derinliklerine yerleşen yazılımdır. Bir rootkit ne kadar düşük (donanıma yakın) katmanda gizlenirse, tespiti o kadar zorlaşır — çünkü tespit araçları genellikle daha üst katmanlarda çalışır ve rootkit onlara yalan söyleyebilir.

Rootkit Türü	Gizlendiği Yer	Tespit Zorluğu
Firmware rootkit	Donanım firmware'i / BIOS	En zor
Hypervisor rootkit	OS'in altına bir hypervisor yerleştirir; OS'i sanal makineye çevirir	Çok zor
Kernel rootkit	İşletim sistemi çekirdeği	Zor
Library rootkit	Sistem kütüphaneleri	Orta
Application rootkit	Uygulama düzeyi	En kolay

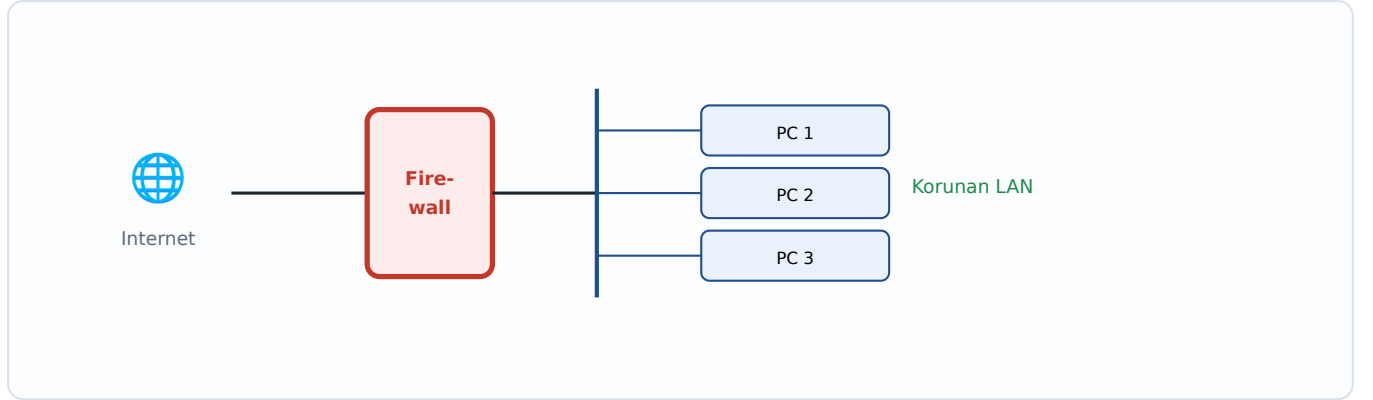


Şekil 8.2 — Bir rootkit'in gizlenebileceği beş yer (🐛 = rootkit): firmware/HW, hypervisor, kernel, kütüphane, uygulama. Aşağı indikçe tespit zorlaşır.

Saldırlara karşı sistemler katmanlı savunmalar (**defense in depth**) kullanır. Bu bölüm, ağ sınırından (firewall) uygulama izolasyonuna (sandbox, jail) ve mobil kod güvenliğine (Java) kadar uzanan savunma cephanesini inceler.

9.1 Güvenlik Duvarları (Firewalls)

Firewall, bir iç ağ (LAN) ile dış dünya (internet) arasına yerleştirilen ve geçen trafiği önceden tanımlanmış kurallara göre **denetleyen/filtreleyen** bir bariyerdir. Donanım firewall'u, korunan ağdaki bilgisayarları doğrudan internete maruz bırakmadan, yalnızca izin verilen trafiğin geçmesine olanak tanır. Filtreleme kaynak/hedef IP, port ve protokol gibi alanlara göre yapılır.



Şekil 9.1 — Bir donanım firewall'u, üç bilgisayarlı bir LAN'ı internetten gelen trafiği filtreleyerek korur.

9.2 Virüs Tarayıcılar (Virus Scanners)

Virus scanner, dosyaları bilinen virüslerin **imzalarına** (signature — karakteristik bayt dizileri) göre tarar. Ancak virüsler tespitten kaçmak için kendilerini gizler: sıkıştırılabilir (compressed), şifrelenebilir (encrypted) ve hatta her bulaşmada görünümünü değiştirebilir (polymorphic). Tanenbaum'un gösterdiği durumlar: (a) temiz program, (b) enfekte program, (c) sıkıştırılmış enfekte program, (d) şifreli virüs, (e) sıkıştırma kodu da şifrelenmiş sıkıştırılmış virüs.

DİKKAT — POLYMORPHIC VİRÜSLER

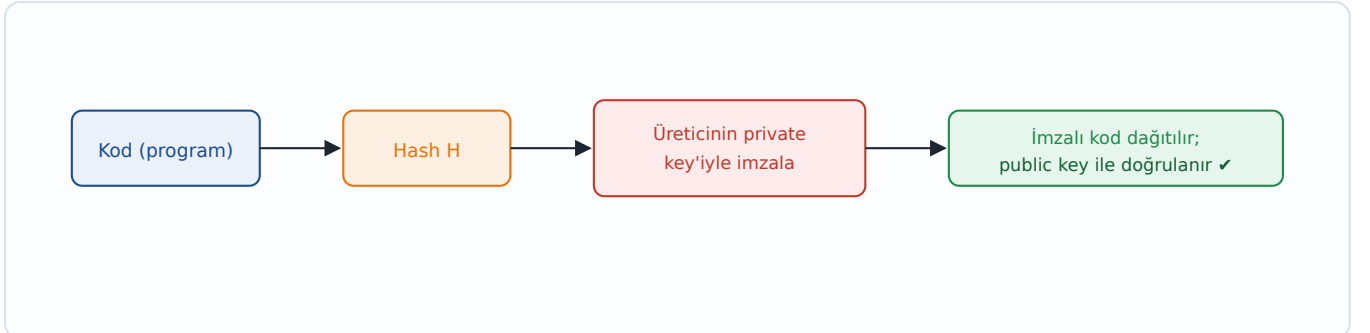
Polymorphic virüs, her kopyasında farklı görünür: gövdesini farklı bir anahtarla şifreler ve her seferinde farklı bir çözücü (decryptor) kodu üretir. Böylece sabit bir imza ile yakalanamaz. Buna karşı tarayıcılar, kodu güvenli bir ortamda **çalıştırıp davranışına** bakan tekniklere yönelir.

9.3 Antivirüs ve Anti-Antivirüs Teknikleri

Teknik	Açıklama
Virus scanners	Bilinen imzalara göre tarama. Hızlı ama yalnızca bilinen virüsleri yakalar.
Integrity checkers	Dosyaların kriptografik checksum'larını saklar; değişiklik olursa enfeksiyon şüphesi.
Behavioral checkers	Programın davranışını (şüpheli sistem çağrıları vb.) izler; imza gerektirmez, yeni virüsleri de yakalayabilir.
Virus avoidance	En iyi savunma: güvenilir OS, az ve denetlenmiş yazılım kurma, yedekleme, güncel yama.

9.4 Kod İmzalama (Code Signing)

Code signing, bir yazılımın **kimden geldiğini** ve **değiştirilmediğini** kanıtlamak için sayısal imza (Bölüm 2.4) kullanır. Üretici, kodun hash'ini kendi private key'ile imzalar; kullanıcının sistemi, üreticinin public key'ile imzayı doğrular. İmza tutmazsa kod ya çalıştırılmaz ya da uyarı verilir. Bu, indirilen yazılımın güvenilir kaynaktan geldiğine dair bir güvence sağlar.



Şekil 9.2 — Code signing'in çalışması: kodun hash'i private key ile imzalanır, kullanıcıda public key ile doğrulanır.

9.5 Jailing (Hapsetme)

Jailing, şüpheli bir programı kısıtlanmış bir ortamda ("jail") çalıştırma tekniğidir. Bir **jailer** süreci, hapsedilen programın yaptığı tüm sistem çağrılarını izler ve izin verilenler dışındakileri engeller. UNIX'teki `chroot` jail bunun klasik örneğidir: programın gördüğü dosya sistemi kökü, gerçek kökün bir alt dizinine kısıtlanır; program bu "hapishaneden" dışarıyı göremez.

9.6 Model Tabanlı Saldırı Tespiti (Model-Based Intrusion Detection)

Bu yaklaşım, bir programın yapması **beklenen** sistem çağrılarının modelini (örneğin bir **system call graph**) önceden çıkarır. Çalışma anında program bu modelden sapan bir çağrı dizisi üretirse (örneğin beklenmeyen bir `exec`), bu bir saldırı işareti sayılır. Mantık: meşru

bir programın davranışı öngörülebilir; buffer overflow gibi bir saldırı ele geçirilen akışı **modelin dışına** taşır.

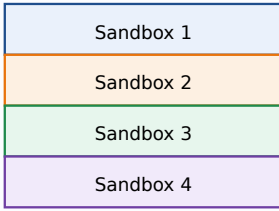
ÖRNEK — SYSTEM CALL GRAPH

Bir program normalde `open → read → write → close` dizisini izliyorsa, çağrı grafiği bu geçişleri içerir. Program aniden bir `open` 'dan sonra `exec("/bin/sh")` çağırırsa (grafikte olmayan bir kenar), IDS bunu anomali olarak işaretler.

9.7 Sandboxing (Yazılımsal İzolasyon)

Bölüm 7'de kavramsal olarak değindiğimiz sandbox'ın verimli bir gerçekleştirilmesi **software fault isolation**'dır. Bellek, eşit boyutlu (örneğin 16 MB) **sandbox**'lara bölünür. Bir programın tüm bellek erişimleri ve atlamaları, kendi sandbox'ının sınırları içinde kalmaya zorlanır. Bunu sağlamak için derleyici/yükleyici, her riskli komuttan önce adresin geçerli sandbox içinde olup olmadığını denetleyen ek komutlar ekler. Böylece program kendi bölgesinin dışına çıkamaz.

(a) 16-MB sandbox'lar



her programa kendi bölgesi

(b) Komut geçerlilik kontrolü

```
// jump R hedefi denetle
R := R AND mask
R := R OR sandbox_base
jump R // artık güvenli
```

Şekil 9.3 — (a) Bellek 16-MB sandbox'lara bölünür. (b) Riskli komutlardan önce adres, sandbox sınırına maskelenerek dışarı çıkması engellenir.

9.8 Yorumlama (Interpretation) ve Mobil Kod

İndirilen mobil kodu (örneğin web applet'lerini) güvenli çalıştırmanın bir yolu, onu doğrudan CPU'da çalıştırmak yerine bir **interpreter** (yorumlayıcı) içinde yürütmektir. Applet bir web tarayıcısı tarafından yorumlanır; tarayıcı her işlemi süzgeçten geçirdiği için applet'in zararlı eylemlerine (dosya silme, ağ erişimi) izin vermez. İzolasyon, donanım yerine yazılımsal yorumlama katmanıyla sağlanır.

9.9 Java Güvenliği (Java Security)

Java, mobil kod güvenliğinin köşe taşıdır. Java kodu doğrudan makine koduna değil, taşınabilir **byte code**'a derlenir ve **JVM** (Java Virtual Machine) içinde çalışır. JVM'in **byte code verifier**'ı, applet'i çalıştırmadan önce belirli kurallara uyup uymadığını denetler:

- Applet **pointer sahteciliği (forge pointers)** yapmaya çalışıyor mu?

- **private** sınıf üyelerine erişim kısıtlarını ihlal ediyor mu?
- Bir tür değişkenini başka bir tür olarak kullanmaya çalışıyor mu?
- **Stack overflow / underflow** üretiyor mu?
- Değişkenleri bir türden diğerine yasadışı biçimde dönüştürüyor mu?

Bu doğrulamayı geçen kod, ayrıca bir **security policy** (örneğin JDK 1.2 ile belirlenen izinler) altında çalışır: hangi dosyalara erişebileceği, hangi ağ bağlantılarını kurabileceği gibi izinler ayrıntılı (fine-grained) olarak tanımlanabilir. Böylece "untrusted" bir applet bile sınırlı ve kontrollü bir ortamda çalışır.

ÖNEMLİ — SAVUNMA KATMANLARI ÖZETİ

- **Ağ sınırı:** Firewall (trafiği filtreler).
- **Dosya/kod denetimi:** Virus scanner, integrity/behavioral checker, code signing.
- **İzolasyon:** Jailing, sandboxing, interpretation, JVM.
- **İzleme:** Model-based intrusion detection.
- Hiçbiri tek başına yeterli değildir → **defense in depth**.

Bu bölüm, güvenliği **resmî olarak değerlendirme ve sertifikalama** çabasını ele alır: erişim kontrolünün neden tek başına yetmediğini, askeri çok seviyeli güvenlik modelini ve Orange Book'tan Common Criteria'ya uzanan değerlendirme standartlarının yükseliş-düşüşünü inceler.

10.1 Erişim Kontrolü Yetmez (Access Controls Won't Do It)

Yaygın bir iddia şudur: "Windows bu kadar çok malware kapıyor çünkü etkili dosya koruması yok." Bu iddia **eksiktir**. Dosya koruması, OS'in kendisinin kirlenmesini (contamination) önleyebilir; ama solucanlar (worms) **kullanıcının kendi yetkileriyle** yayılabilir ve hiçbir OS korumasını ihlal etmezler. İki tarihsel örnek bunu kanıtlar:

- **IBM Christmas Card "Virus" (1987):** E-posta ile gönderilen bir Trojan Horse shell script'ine dayanıyordu — kullanıcı çalıştırdı, yetkisiyle yayıldı.
- **Morris Internet Worm (1988):** Çok-açıklı (multi-exploit), çok-platformlu bir solucandı ve **hiçbir OS korumasını ihlal etmedi**; meşru servislerin zayıflıklarını kullandı.

DİKKAT — TEMEL İÇGÖRÜ

Güçlü erişim kontrolü güvenliğin **gerekli** bir parçasıdır, ama yeterli değildir. Saldırıların çoğu, kullanıcının **meşru yetkileri** içinde gerçekleşir. Bu, Bölüm 1'deki "gerekli ama yeterli değil" temasının doğrudan devamıdır.

10.2 Sertifikalı Sistemler ve Orange Book

1980'lerin başında ABD Savunma Bakanlığı, **Orange Book** olarak bilinen *DoD Trusted Computer System Evaluation Criteria*'yı (TCSEC) yayımladı. Orange Book, sistemleri güvenlik seviyelerine göre derecelendiriyordu: **D** (güvenlik yok) seviyesinden **A1** (formally verified — matematiksel olarak doğrulanmış) seviyesine kadar. Üst seviyeler hem daha fazla **özellik (features)** hem de — daha da önemlisi — daha yüksek **güvence (assurance)** talep ediyordu.

Seviye	Anlamı
D	Minimal koruma — güvenlik yok.
C1 / C2	Discretionary protection (DAC), kullanıcı bazlı denetim ve loglama.
B1 / B2 / B3	Mandatory protection (MAC), etiketli güvenlik, artan yapı ve güvence.
A1	Verified design — formal yöntemlerle doğrulanmış en yüksek güvence.

10.3 Askeri Sınıflandırma Modeli (Military Classification Model)

Orange Book'un arkasındaki mantık, askeri gizlilik modeline dayanır: **belgeler** belirli bir seviyede sınıflandırılır, **kişiler** belirli yetkilere (clearance) sahiptir ve bir kişi yalnızca **yetkili olduğu** belgeleri görebilir. Bu, Bell-La Padula'nın (Bölüm 4) gerçek dünyadaki kaynağıdır.

Sınıflandırmalar (Classifications)

- **Levels (seviyeler):** Confidential < Secret < Top Secret.
- **Compartments (bölmeler):** Crypto, Subs, NoForn ("NoForn" = No Foreign Nationals — yabancı uyruklu yok).
- **Erişim kuralı:** Bir belgeyi okumak için, kişinin (1) **en az o seviye kadar** clearance'a sahip olması **ve** (2) belgenin **her compartment'**ı için yetkili olması gerekir.
- Bunu destekleyen sistemlere **multi-level security** (çok seviyeli güvenlik) sistemleri denir.

ÖRNEK — "KİM HANGİ DOSYAYI OKUYABİLİR?" (SLAYTTAKİ SORU)

Kişiler: Pat → (Secret, Subs); Chris → (Top Secret, Planes).

Dosyalar:

- warplan → Top Secret {Troops, Subs, Planes}
- runway → Confidential {Planes}
- sonar → Top Secret {Subs}
- torpedo → Secret {Subs}

Çözüm:

- **warplan'ı kimse okuyamaz:** Pat'in seviyesi yetmez ve Troops/Planes yetkisi yok; Chris'in de Subs/Planes... aslında Chris'in Subs ve Troops yetkisi yok → okuyamaz.
- **runway:** Chris okuyabilir (Top Secret ≥ Confidential, Planes yetkisi var). Pat okuyamaz (Planes yetkisi yok).
- **sonar (Top Secret, Subs):** Pat okuyamaz — Subs yetkisi var ama yalnızca Secret seviyesinde (Top Secret > Secret).
- **torpedo (Secret, Subs):** Pat okuyabilir — Secret seviyesi yeterli ve Subs yetkisi var.

SINAV İPUCU

Bu tip "clearance vs classification" sorularında **iki koşulu birden** kontrol edin: (1) seviye: kişinin clearance'ı ≥ belgenin seviyesi mi? (2) compartment: kişi, belgenin **tüm** bölmeleri için yetkili mi? İkisi de sağlanmazsa erişim **yok**. En sık yapılan hata, yalnızca seviyeye bakıp compartment'ı unutmaktır.

10.4 Güvence (Assurance)

Assurance, "OS'in doğru olduğuna neden inanıyoruz?" sorusunun yanıtıdır. Geliştiriciler işlerini ne kadar iyi yaptı? Kötü niyetli geliştiriciler arka kapı (back door) yerleştirdi mi? Orange Book'un üst seviyeleri daha yüksek assurance talep ediyordu: iyi dokümantasyon, yapısal tasarım, formal test planları ve hatta **güvenlik soruşturmasından geçmiş (security-cleared) programcılar**.

ÖNEMLİ — FEATURES VS ASSURANCE

Features (özellikler): Sistemin *ne yaptığı* (şifreleme, MAC, loglama...).

Assurance (güvence): O özelliklerin *doğru çalıştığına ne kadar güvenebileceğimiz*. Orange Book'un en büyük fikri, yüksek güvenliğin yalnızca özellik değil, **kanıtlanabilir doğruluk** gerektirdiğiydi.

10.5 Orange Book'un Kaderi ve Common Criteria

Orange Book yalnız değildi: İngiltere ve Batı Avrupa **ITSEC**'i, Kanada kendi sürümünü, ABD ise **Federal Criteria**'yı üretti. Sonunda bunların hepsi tek bir uluslararası standartta — **Common Criteria** (CC) — birleştirildi.

Common Criteria'nın temel yeniliği, Orange Book'un karıştırdığı **features ve assurance**'ı ayırmasıdır. Ayrıca değişken **Protection Profile**'lara izin verir:

- **Protection Profile:** Sistemin *ne başarması gerektiğini* söyler (hedef).
- **Feature list:** Bunu *nasıl yaptığını* söyler.
- **Assurance level:** Buna ne kadar *güvenmeniz gerektiğini* söyler.

10.6 Kötü Modeller, Satış Yok (Bad Models, No Sales)

Orange Book ticari olarak başarısız oldu ve nedenleri öğreticidir:

- Güvenlik modeli ticari açıdan ilgi çekici değildi.
- Bir sistemi değerlendirmek **zaman alıcı ve pahalıydı**.
- Değerlendirilmiş sistemler ticari olanların çok gerisinde kalıyor ve daha pahalı oluyordu.
- 1980'lerin **timesharing** sistemleri için tasarlanmıştı; **networking** gibi "küçük" şeyler kapsam dışıydı.
- Aynı sorunların çoğu **Common Criteria** sistemlerini de etkiledi.

DİKKAT — ALINACAK DERS

Güvenlik bir standart belgesiyle "sertifikalanarak" çözülemez; eğer süreç pahalı, yavaş ve gerçek kullanım senaryolarından (networking gibi) kopuksa, piyasa onu görmezden gelir. Güvenlik, kullanılabilir ve güncel olmak zorundadır.

Önleme her zaman mümkün değildir; bu yüzden **tespit ve sorgulama** kritiktir. Bu son bölüm, loglamanın rolünü, ne kadar log tutulması gerektiğini ve dersin nihai mesajını — gerçek güvenlik sorununun OS'te değil, **uygulamalarda** olduğunu — ele alır.

11.1 Loglama (Logging)

Loglama üç temel soruya yanıt verir: (1) Sistemlerinizde **ne oluyor?** (2) Bir sızma (penetration) olursa **bunu bilecek misiniz?** (3) **Nasıl olduğunu** sonradan çözebilecek misiniz? Loglar, saldırıyı anlık olarak engellemese de, tespit ve adli analiz (forensics) için vazgeçilmezdir.

11.2 Vaka: Shadow Hawk

Shadow Hawk, 1987'de FBI tarafından yakalanan bir saldırgandır. Bell Labs makinelerine girmiş ve çeşitli AT&T sistemlerinden (Naperville, Murray Hill) C kaynak kodları indirirken tespit edilmiştir.

Nasıl Tespit Edildi?

Shadow Hawk, başka makinelerden `/etc/passwd` dosyalarını almak için **uucp**'yi (UNIX'le gelen bir çevirmeli dosya transfer/e-posta sistemi) kullanmaya çalıştı. **uucp tüm dosya transfer isteklerini logluyordu.** Murray Hill'deki bazı kişilerin, log dosyalarını şüpheli etkinlik için **otomatik tarayan** işleri vardı — saldırgan bu loglar sayesinde yakalandı.

ÖNEMLİ — SHADOW HAWK'IN DERSİ

Loglamanın değeri, yalnızca log **tutmakla** değil, logları **aktif olarak izlemekle/analiz etmekle** ortaya çıkar. Kimsenin bakmadığı bir log dosyası bir saldırıyı durdurmaz.

11.3 Ne Loglanmalı? (What to Log?)

Her şeyi mi loglayalım? Bunun ciddi ödünleşimleri (trade-off) vardır:

- Çok fazla depolama gerektirebilir (gerçi disk ucuzdur).
- Ciddi bir **gizlilik (privacy)** riski doğurur.
- Bu kadar veriyi **işleyebilir misiniz?** (Analiz kapasitesi.)
- Ama — **güvenlik açısından hassas olaylar mutlaka loglanmalıdır.**

11.4 Solaris Basic Security Module (BSM)

Solaris BSM, kapsamlı loglama yapabilen bir örnektir. Loglayabildiği kategoriler arasında login/logout, `ioctl`, dosya yazma, ağ olayları, mount/unmount, `fork`, `exec` ve daha fazlası bulunur. Log dosyalarını korumak için büyük özen gösterilir (saldırgan izini silememelidir). Yine de aynı soru geçerlidir: **bu kadar veriyi işleyebilir misiniz?**

11.5 Sonuç: "It's the Application"

Dersin nihai mesajı keskindir: **bir işletim sisteminin gerçek amacı belirli uygulamaları çalıştırmaktır**. Dolayısıyla mesele OS'in ne kadar güvenli olduğu değil, **uygulamaların** ne kadar güvenli olduğudur. Tekrar hatırlayalım: çoğu solucan OS güvenliğini ihlal etmez — uygulama zafiyetlerinden faydalanır.

11.6 Asıl Zorluk (The Challenge)

Kullanışlı ve güvenli bir OS, **güvenli uygulama yazmayı kolaylaştırmalıdır**. Bu, kullanışlı sandbox'lar gibi araçlar demektir. Ayrıca daha esnek bir izin modeli gereklidir:

- **DAC** (Discretionary Access Control) çoğu zaman **fazla basittir** (Trojan, sahibin yetkisiyle her şeyi yapabilir).
- **MAC** (Mandatory Access Control) çoğu zaman **fazla kısıtlayıcıdır** (esnek günlük kullanım zorlaşır).
- İhtiyaç: ikisinin arasında, hem güvenli hem kullanılabilir bir orta yol.

11.7 Henüz Sahip Değiliz (We Don't Have Them)

DİKKAT — GERÇEKÇİ SONUÇ

Bu kriterleri karşılayan **hiçbir ticari OS yoktur**. Üstelik çoğu uygulama, elimizdeki mevcut (sınırlı) olanaklara göre tasarlanmıştır. Her zaman **buggy code** (hatalı kod) olacaktır. Hüner, saldırılara büyük ölçüde direnen ve **önemli varlıkları (assets)** koruyan bir uygulama+OS bileşimi inşa etmektir. Mükemmel güvenlik bir hedef değil, sürekli bir **denge ve risk yönetimi** pratiğidir.

ÖNEMLİ — DERSİN BÜYÜK RESMİ (TÜM BÖLÜMLERİN ÖZETİ)

- **Hedef:** Confidentiality, Integrity, Availability (CIA).
- **Temel:** Authentication → identity → access control (ACL / capabilities).
- **Modeller:** Bell-La Padula (gizlilik), Biba (bütünlük); reference monitor + TCB.
- **Saldırıları:** Trojan, login spoofing, buffer overflow, code injection, malware, rootkit.
- **Savunma:** Firewall, scanner, code signing, sandbox/jail, IDS, logging — **defense in depth**.
- **Gerçek:** OS koruması gerekli ama yeterli değil; asıl risk **uygulamalardadır**.

Bu rehber, CSE 312 İşletim Sistemleri dersi — Ders 9 (Security) sunumundaki 102 slaytın tamamını kapsamaktadır. Hazırlık:
Tanenbaum & Bos, *Modern Operating Systems*, 5. Baskı temel alınarak. Başarılar! 🙏